I am completing an individual college Lab 4: VibeCoding assignment.

Before making any changes, read the complete README.md and the Lab_4_VibeCoding_Student_handout.pdf in this repository.

INSPECTION ONLY. Do not modify files, install packages, create commits, or push anything.

Report:
1. All assignment requirements and deliverables from the handout.
2. All four tasks and expected behaviors from the README.
3. The project structure, dependencies, and how the original game works.
4. The current Git branch, remotes, and working-tree status.
5. A plan for implementing and testing each task separately, with a separate Git commit for each verified task.
6. How to preserve the actual Claude Code conversation for the required chat-history submission.

Important constraints:
- The original game has already been run, and I have recorded a before video.
- Do not modify the original code until I approve.
- Do not create a pull request to SETAPESU26.
- Push the finished project only to my personal GitHub repository.
- Do not claim that any task is complete without testing it.
- If you cannot read the PDF, tell me explicitly.

Provide your findings and then stop for my approval.

● Everything below comes from reading the files and git metadata. I haven't modified, installed, committed or pushed anything. I could read the PDF; it's one page.

1. Handout requirements (Lab_4_VibeCoding_Student_handout.pdf)

- Individual assignment, about 60 minutes. The goal is to fix the code and add features using an AI tool, ideally in three or four prompt attempts per item.
- Steps:
  a. Clone or fork the assigned repo and read the README.

   b. Run the game and record a 10-second before video. You've already done this.
   c. Use prompts to fix the broken code.
   d. Use prompts to add each README feature, one at a time.
   e. Record a 10-second after video.
   f. Push to your own repo, with a separate commit for each task.
   g. Do not raise a PR to SETAPESU26.
 - Deliverables go "to your Lab repo under a Lab-4 folder":
   - (a) the before and after videos
   - (b) the updated code
   - (c) the chat history exported as a doc or PDF
 - The README's checklist asks for a "Chat/LLM page link with complete chat history" instead of a doc or PDF. Delivering both would cover each wording.

 2. README tasks

Task	Requirement
1. Collision detection	The clickable area has to match the drawn, shrinking circle. Late clicks outside the small circle must not count as hits.
2. Game over	When the timer reaches 0, show a screen with the final score and accuracy, then wait for input. It shouldn't just print to the console.
3. Replay	After Game Over, let the player choose Easy, Medium or Hard to play again, or exit. Difficulty changes the target's lifespan and size.

4. Sound	Add sounds for a hit, a miss (including a target that times out) and the round ending.

The README also lists behaviour that must keep working:
- One random target at a time, which shrinks while it's on screen.
- A hit spawns a new target immediately.
- A click anywhere else, or a target timing out, counts as a miss.
- The timer, score and live accuracy are always visible.
- The round ends at 0.

3. Project structure and how the original game works

main.py – pygame init, 700x500 window, 60 FPS loop: events → engine.handle_input/update/render
game/game_engine.py – GameEngine: spawning, clicks, frame-based 30s timer, hits/misses/score, HUD
game/target.py – Target(x, y, base_radius=40, min_radius=12, lifespan_frames=90)
requirements.txt – pygame (unpinned)
README.md
game/__pycache__/*.pyc – compiled files, committed by mistake

- Environment: Python 3.12.7 and pygame 2.6.1 are already installed. There's no .gitignore, no sound files and no tests.
- Target: age goes up by 1 each frame. visual_radius() shrinks linearly from 40 to 12 over 90 frames (1.5 seconds), and expired() becomes true at 90 frames.
- Engine:
  - The timer is 30 * 60 frames, so it depends on the frame rate.
  - A click inside contains_point scores a hit and spawns a new target. Any other click is a miss.
  - An expired target counts as a miss and is replaced.
  - When the timer reaches 0, game_over = True and input is ignored. render prints the result once to the console, and the frozen target and HUD stay

on screen.
- Task 1 bug: target.py:31 checks the click against base_radius (always 40) instead of visual_radius(). The code comments even point this out.
- Things to watch:
  - render draws int(visual_radius()), so the hit test should use the same radius to match exactly.
  - Clicks are handled before update(), so a click is tested against the radius that was on screen at that moment. That's correct as it is.

4. Git state

- Branch: main, up to date with origin/main. There's one commit, a08501a Initial Target Aim Trainer starter.
- Remote: origin points to https://github.com/SETAPESU26/47_target-aim-trainer.git (fetch and push). That's the class repo, not yours.
- Working tree:
  - The two tracked game/__pycache__/*.pyc files show as modified. Running the game regenerated them.
  - Lab_4_VibeCoding_Student_handout.pdf is untracked.

5. Proposed plan (waiting for your approval)

Setup (no code changes):
1. Rename origin to upstream and disable pushing to it (git remote set-url --push upstream DISABLED), so nothing can reach SETAPESU26 by accident.
2. Add your personal GitHub repo as origin. I need its URL from you.
3. Restore the .pyc files and keep the handout PDF out of the commits.
4. Optional: a housekeeping commit that adds a .gitignore for __pycache__/ and stops tracking the .pyc files.

Testing approach: for each task I'll write a headless test script in my scratchpad, outside the repo. It runs pygame with SDL_VIDEODRIVER=dummy and SDL_AUDIODRIVER=dummy and drives the engine directly. After that you run python main.py to check by hand. I'll only commit once both have passed, and I'll report the actual test output.

Task: 1
Change: contains_point uses the drawn radius (int(visual_radius()))
Verification: A click at distance 30 on a fresh target is a hit. At age 80 the

same click is a miss. Edge cases at the exact radius.
Commit: Task 1: Match hit area to shrinking target radius
Task: 2
Change: Draw a game-over overlay (title, final score, accuracy, hits/misses, prompt) and remove the console print. Clicks are ignored; Esc/Q or closing the window quits.
Verification: Force the timer to 0. Check that the state flips, the overlay renders without errors and score/accuracy stop changing. Save a screenshot for you to look at.
Commit: Task 2: Add game over screen
Task: 3
Change: A difficulty table, passed to Target, plus reset(difficulty). On the game-over screen: 1/E = Easy, 2/M = Medium, 3/H = Hard, Q/Esc = exit.
Suggested values: Easy 50→20 px, 150 frames; Medium 40→12, 90 (the current game); Hard 30→8, 60.
Verification: After a reset, the score, timer and target are fresh and the targets have the chosen size and lifespan. Each key works and Q exits.
Commit: Task 3: Add replay with difficulty selection
Task: 4
Change: Generate short tones at startup with pygame.mixer.Sound(buffer=...), so
there's no new dependency and no audio files. Play them on a hit, a miss or timeout, and once when the round ends. If audio isn't available, the game keeps running without sound.
Verification: Stub the sound objects and check that each event plays the right

sound the right number of times, including a single play at game over. Then you listen during a manual run.
Commit: Task 4: Add sound effects for hit, miss, and round end

Finishing:
1. Record the after video.
2. Push only to your personal origin, and check git remote -v before pushing.
3. No PR anywhere.

Open question: the handout says the deliverables go in a Lab-4 folder in your Lab repo. Is that this game repo with a Lab-4/ folder for the videos and chat PDF, or a separate lab repo? I'll follow whichever you choose.

6. Saving this conversation for the chat-history deliverable

- Most complete copy: Claude Code automatically saves this session as a JSONL file under C:\Users\rohit\.claude\projects\C--Users-rohit-47-target-aim-trainer\, named after the session ID. Keep a copy before you clean anything up.
- For the doc/PDF submission: run /export at the end. It saves the conversation as a text file or copies it to the clipboard. Paste it into Word or Google Docs and export as PDF. Put that PDF in Lab-4/ and commit it last.
- Things to avoid until you've exported:
  - Don't run /clear.
  - Avoid /compact.
  - If you close the terminal, reopen the session with claude --resume or claude --continue instead of starting a new one.
- If you need a link: the README asks for a page link, but a local Claude Code session doesn't produce one. The exported PDF meets the handout's "doc/pdf" requirement. You could also ask your instructor whether that's acceptable instead of a link.

To proceed, I need your approval of this plan, your personal repo URL, and your answer on where the Lab-4 folder goes.

Update to my submission setup:

My existing personal GitHub Lab repository has now been cloned to:
C:\Users\rohit\PES1UG24AM229_SE_LAB

Its GitHub remote is:
https://github.com/rohithkarning-debug/PES1UG24AM229_SE_LAB.git

The final assignment deliverables must be placed in:
C:\Users\rohit\PES1UG24AM229_SE_LAB\Lab-4

The aim-trainer project remains at:
C:\Users\rohit\47_target-aim-trainer

Important instructions:

1. Keep the two repositories separate.
2. Do not push anything to SETAPESU26.
3. Preserve the existing task-by-task implementation plan and separate commits for the four tasks.
4. Preserve my actual Claude Code conversation for the required chat-history deliverable.
5. Do not modify game files yet.
6. Do not change Git remotes, restore files, create commits, or push anything yet.

First, acknowledge these paths and explain the safest way to continue from the inspection report you already produced. Then wait for my approval.

Ran 1 shell command (ctrl+o to expand)

● Paths confirmed

I checked both repos with read-only commands and changed nothing.

	Aim-trainer project	Personal Lab repo
Path	C:\Users\rohit\47_target-ai m-trainer	C:\Users\rohit\PES1UG24AM229_S E_LAB
Remote	origin → SETAPESU26 (class repo)	origin → rohithkarning-debug/P ES1UG24AM229_SE_LAB
Branch / state	main; the 2 regenerated .pyc files are modified and the handout PDF is	main, up to date, clean working tree; last commit is 96c1458 Completed Lab 3

	untracked	
Deliverables	—	Lab-4\ exists but is empty (and untracked, since Git ignores empty folders)

Safest way to continue

The commits you submit have to be in your personal repo. The aim-trainer repo can only push to SETAPESU26. So I recommend doing all the commits in the Lab repo and never committing or pushing from the aim-trainer repo.

1. The aim-trainer repo stays untouched as the original. No commits, no pushes and no remote changes there. That rules out any push to SETAPESU26, and you keep a clean original to compare against.
2. Baseline commit in the Lab repo. Copy only the source files (main.py, requirements.txt, README.md, game/game_engine.py, game/target.py) into Lab-4\target-aim-trainer\. I wouldn't copy .git, __pycache__ or the handout PDF. Add a small .gitignore for __pycache__/, then commit: Lab 4: Add original Target Aim Trainer starter code. That way each task commit's diff shows only that task's change.
3. Four task commits, one per task, each tested before it's committed. This is the same plan as before, applied to the files in Lab-4\target-aim-trainer\:
   - Headless test scripts live in my scratchpad, outside both repos.
   - You also run python main.py and check by hand.
   - I commit only after both checks pass, and report the actual output.
4. Deliverables commit: the before and after videos plus the chat-history PDF, for example Lab-4\videos\before.mp4, Lab-4\videos\after.mp4 and Lab-4\chat_history.pdf. Check the video sizes first: GitHub rejects files over 100 MB and warns above 50 MB. A 10-second clip is normally well under that.
5. Push: before pushing, I'll check git remote -v in the Lab repo, then push main to rohithkarning-debug/... only. No PR.

Chat history: this session is saved automatically at C:\Users\rohit\.claude\projects\C--Users-rohit-47-target-aim-trainer\<session-id>.jsonl.
- Stay in this session the whole time. Don't use /clear or /compact, and if the terminal closes, reopen with claude --resume.
- At the end, run /export, convert the export to PDF, and commit it last so it covers the whole conversation.
- I can also save a raw copy of the .jsonl into Lab-4\ as a backup if you'd like.

What I need before starting

1. Your approval of the copy-into-the-Lab-repo approach, including the baseline commit and the Lab-4\target-aim-trainer\ subfolder.
2. Where your before video is saved, so it can go into Lab-4\ later.
3. Whether to leave the modified .pyc files in the aim-trainer repo as they are. They're harmless if nothing is committed there, so I'd leave them.

We have agreed on the following plan for my Lab 4 VibeCoding assignment. Please follow it carefully.

1. Repositories and Git safety

My original assigned game repository is located at:
`C:\Users\rohit\47_target-aim-trainer`

My personal Lab repository is located at:
`C:\Users\rohit\PES1UG24AM229_SE_LAB`

My personal GitHub repository is:
`https://github.com/rohithkarning-debug/PES1UG24AM229_SE_LAB.git`

The original game repository must remain untouched from a Git perspective. Do not commit, push, change its remotes, or create branches in it. Leave its current modified `.pyc` files and untracked handout PDF alone.

All assignment commits must be made inside my personal Lab repository. Before any push, verify that its Git remote points to my personal repository above. Never push to `SETAPESU26` and never create a pull request to the university

repository.

2. Required project structure

Inside my personal Lab repository, use this structure:

`Lab-4\`
- `target-aim-trainer\` — the game source code and its README/requirements.
- `videos\` — the before and after recordings.
- `chat_history.pdf` — the complete exported Claude Code conversation.

I will upload the before and after videos myself. Do not search for, copy, generate, or modify the recordings.

Ensure that `Lab-4\videos\` is tracked by Git. Since Git does not track empty folders, create a small `README.md` inside `Lab-4\videos\` explaining that I will place the required before and after recordings here. I will upload the videos later.

Do not copy `.git`, `__pycache__`, generated `.pyc` files, or the handout PDF into the game project. Copy only the required source files and supporting project files.

3. Assignment requirements

Follow the Lab 4 handout and the assigned game's README. The assignment requires:
- Fix the broken code.
- Add every feature specified in the README, one task at a time.
- Record before and after videos (I will handle these).
- Push the updated code to my personal repository.
- Maintain separate commits for each task.
- Export the complete AI chat history as a DOC or PDF.

The four game tasks are:

Task 1 — Collision detection:
Make the clickable area match the target's current visible shrinking radius. Clicks outside the visible circle must not count as hits.

Task 2 — Game Over screen:
Display the final score and accuracy on the game screen when time runs out, rather than only printing the results to the terminal.

Task 3 — Replay and difficulty:
Allow the player to replay with Easy, Medium, or Hard difficulty, or exit.
Difficulty must change target size and lifespan.

Task 4 — Sound feedback:
Add sound feedback for a hit, a miss (including target timeout), and the end of the round. Handle audio initialization failures gracefully.

Preserve the original gameplay and existing functionality, including target spawning, shrinking targets, score, timer, and live accuracy.

4. Required workflow and commits

Work incrementally and explain each step in simple language.

First:
1. Inspect the Git status and remotes of both repositories without modifying them.
2. Verify the destination paths and confirm the personal repository is the one that will receive all commits.
3. Copy the original starter source files into `Lab-4\target-aim-trainer\`.
4. Add an appropriate `.gitignore` to prevent Python cache files from being committed.
5. Ensure `Lab-4\videos\README.md` exists.
6. Review the staged changes carefully.
7. Create one baseline commit for the original starter code and supporting folder structure, with a message such as `Lab 4: Add original Target Aim Trainer starter code`.
8. Verify the commit and show me the result.

After the baseline commit, STOP and wait for my confirmation. Do not start Task 1 until I confirm.

For the remaining work, use separate commits:
- One commit for Task 1.
- One commit for Task 2.

- One commit for Task 3.
- One commit for Task 4.
- One final commit for the chat-history export and any remaining submission documentation.

Test each task before committing it. Use automated/headless tests where appropriate, then ask me to run `python main.py` and manually verify the game behavior. Do not claim a test passed unless it was actually run and passed. If a test fails, investigate and fix the issue before committing.

5. Chat history preservation

Preserve this Claude Code session so I can export the complete conversation at the end. Do not use `/clear` or `/compact`. If the session needs to be resumed, explain how to resume it.

At the end of all four tasks, help me export the complete conversation using Claude Code's `/export` command and save it as `Lab-4\chat_history.pdf`. Do not fabricate or reconstruct a fake conversation transcript.

6. Push safety

Do not push anything yet.

Once all tasks and deliverables are ready, check `git status`, review the commit history, and verify `git remote -v` from the personal Lab repository. Only push to:

`https://github.com/rohithkarning-debug/PES1UG24AM229_SE_LAB.git`

Never push to `SETAPESU26`. Do not create a pull request.

Your immediate next action

Start with inspection and the baseline setup only. Do not implement any of the four tasks yet. Show me the planned files to copy and the Git changes before committing. Create and verify the baseline commit, then stop and wait for my confirmation.

● Step 1: check both repos again without changing anything.

Ran 1 shell command (ctrl+o to expand)

● No changes in either repo since last check, and Lab-4 is still empty, so I'll copy main.py, requirements.txt, README.md, and the game engine/target files into Lab-4\target-aim-trainer, plus create a new .gitignore and a videos README, skipping .git, caches, and the handout PDF.

Ran 1 shell command (ctrl+o to expand)

● The copies don't match byte-for-byte. I think this is just line endings (Windows checks files out with CRLF, while Git stores LF), but I'm checking rather than assuming.

Ran 1 shell command (ctrl+o to expand)

● The difference was just line endings—content matches the original, and since the Lab repo has core.autocrlf=true, Git will store them as LF just like the class repo. Now I'll stage only Lab-4 and review the changes.

Ran 1 shell command (ctrl+o to expand)

● Exactly 7 files are staged, all under Lab-4, and there are no cache files or PDFs. As a final check, the staged game files should have the same Git object IDs as the originals in the class repo:

Ran 1 shell command (ctrl+o to expand)

● All 5 game files match the original starter code exactly. Step 7: create the baseline commit.

Ran 1 shell command (ctrl+o to expand)
└ PostToolUse:Bash says: Tip: Run /ultrareview before you push to catch bugs with a cloud-based multi-agent review — 3 free reviews left.

● The baseline commit 4e04a46 is in your personal Lab repo, and nothing has been pushed.

What I did:
1. Checked both repos. The game repo still points to SETAPESU26 and is exactly

as you left it: the two modified .pyc files, the untracked handout PDF and commit a08501a. I didn't touch it. The Lab repo points to rohithkarning-debug/PES1UG24AM229_SE_LAB and was clean before I started.
2. Copied the 5 starter files into Lab-4\target-aim-trainer\: main.py, requirements.txt, README.md and the two files in game\. No .git, __pycache__, .pyc files or handout PDF.
3. Confirmed the copies are the original code. A first byte-for-byte comparison showed a difference, which turned out to be only Windows line endings. The final check matched each file's Git ID against the original commit, and all 5 are identical.
4. Added Lab-4\target-aim-trainer\.gitignore, so Python cache files can't be committed.
5. Added Lab-4\videos\README.md, which says you'll add the recordings there. I didn't look for or touch any videos.
6. Reviewed the staged changes: 7 new files, all inside Lab-4, with no cache files or PDFs.
7. Committed: 4e04a46 Lab 4: Add original Target Aim Trainer starter code. The author is Rohith Karning, and it includes the Co-Authored-By: Claude line. The Lab repo now shows ahead 1 of your GitHub copy, meaning it isn't pushed yet.

Saving this session for the chat history:
- Claude Code files sessions by the folder they started in. This one started in C:\Users\rohit\47_target-aim-trainer.
- If the terminal closes, cd into that folder and run claude --resume, then pick this session.
- Don't use /clear or /compact.

From Task 1 on, all code changes will be made only in C:\Users\rohit\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\.

I'll wait for your go-ahead before starting Task 1.

Confirmed, proceed with Task 1

Read 1 file (ctrl+o to expand)

● I'll fix the click check, which currently uses the fixed 40px starting radius instead of the actual drawn radius. I'm adding a drawn_radius() method and

using it in both the drawing and click-detection code to keep them in sync.

● Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\target.py)
⎿ Added 10 lines, removed 7 lines

```
21          t = min(1.0, self.age / self.lifespan_frames)
22          return self.base_radius - (self.base_radius - self.min_
      radius) * t
23
24 +    def drawn_radius(self):
25 +        # The whole-pixel radius actually drawn on screen. Both
      + render()
26 +        # and contains_point() use this so the clickable area a
      +lways
27 +        # matches the visible circle.
28 +        return int(self.visual_radius())
29 +
30      def contains_point(self, x, y):
25 -        # NOTE: hit-testing always uses the target's starting r
      -adius
26 -        # (base_radius), not its current, shrunken on-screen ra
      -dius
27 -        # (visual_radius()). The circle drawn in render() shrin
      -ks as
28 -        # the target ages, but the clickable area never does, s
      -o a
29 -        # late click well outside the small visible circle can
      -still
30 -        # register as a hit. See Task 1 in the README.
31 -        return math.hypot(self.x - x, self.y - y) <= self.base_
      -radius
31 +        # Hit-test against the current, shrunken on-screen radi
      +us rather
32 +        # than the starting radius, so late clicks outside the
      +visible
33 +        # circle count as misses.
34 +        return math.hypot(self.x - x, self.y - y) <= self.drawn
      +_radius()
```

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)

└ File must be read first

Read 1 file (ctrl+o to expand)

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
└ Added 1 line, removed 1 line
71 return round(100 * self.hits / total, 1)
72
73 def render(self, screen):
74 - r = int(self.target.visual_radius())
74 + r = self.target.drawn_radius()
75 pygame.draw.circle(screen, RED, (self.target.x, self.ta
rget.y), r)
76 pygame.draw.circle(screen, WHITE, (self.target.x, self.
target.y), r, 2)
77

● Now a headless test script. It lives in my scratchpad, outside both repos.

● All 19 checks passed — the only caveat is minor edge-pixel differences (up to 1px) between drawn and clickable areas, far smaller than the original 25px+ bug. Now I'll run the same late-click scenario against the original repo code (read-only, with bytecode writing disabled) to confirm the test actually catches the bug.

Ran 1 shell command (ctrl+o to expand)

● Confirmed the fix works: a click 30px from a 15px circle's center now correctly registers as a miss, whereas the original buggy code counted it as a hit. The class repo remains unchanged, and here's the full diff ready to commit in the Lab repo.

Ran 1 shell command (ctrl+o to expand)

● I've fixed Task 1 and all 19 automated checks passed, but I haven't committed it. I need you to try it in the game first.

What was wrong: the game drew a circle that shrinks from 40 px down to 12 px, but the click check always used the full 40 px. A late click well outside the

small red circle still counted as a hit. I confirmed this on the original code: with the circle shrunk to 15 px, a click 30 px from the centre counted as a hit.

The fix (in the Lab repo only):
- game/target.py has a new drawn_radius() that gives the radius actually being drawn, and the click check now uses it.
- game/game_engine.py draws the circle using that same value, so the drawing and the clickable area can't drift apart again.

What the tests covered:
- Clicks at the edge and just outside it, on a fresh target, a half-shrunk one and a fully shrunk one.
- The original bug case, a late click 30 px away, is now a miss.
- Existing behaviour still works: a hit gives a point and a new target, a miss leaves the target in place, accuracy is right, and a target that runs out of time counts as a miss.
- The clickable area matches the drawn pixels except for single edge pixels, never more than 1 px off.

Please check it by hand:
cd C:\Users\rohit\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer
python main.py
1. Wait until a target has shrunk, then click just outside the red circle, inside where the original 40 px circle would have been. That should count as a miss: accuracy drops and the target stays.
2. Clicking inside the red circle should still be a hit: score +1 and a new target appears.
3. The timer, score, accuracy and timed-out misses should work as before. At 0 the round still only prints to the terminal, because that's Task 2.

Running the game will create a __pycache__ folder, and the new .gitignore keeps it out of commits.

If it behaves correctly, tell me and I'll make the commit Task 1: Match hit area to shrinking target radius, then stop before Task 2.

Please commit the Task 1 changes as a separate commit with a clear message,

verify the commit and Git status, and show me the result. Do not push anything yet, and do not start Task 2 until I confirm.

Ran 1 shell command (ctrl+o to expand)

● Task 1 is committed as 03cdc5f in your Lab repo. Nothing has been pushed.

The commit:
- Message: Task 1: Match hit area to shrinking target radius
- Author: Rohith Karning, with the Co-Authored-By: Claude line.
- Files: only game/target.py and game/game_engine.py (11 lines added, 8 removed).

History:
03cdc5f Task 1: Match hit area to shrinking target radius
4e04a46 Lab 4: Add original Target Aim Trainer starter code
96c1458 Completed Lab 3

Status: the working tree is clean. It's 2 commits ahead of your GitHub copy, meaning they aren't pushed yet. The remote is still only rohithkarning-debug/PES1UG24AM229_SE_LAB.

The 19 automated checks passed before this commit. You haven't told me how the manual python main.py check went. If it showed a problem, let me know and I'll fix it in a follow-up commit.

I'll wait for your go-ahead before starting Task 2.

Confirmed, proceed with Task 2

Read 2 files (ctrl+o to expand)

● For Task 2, I'll have the engine show a semi-transparent "Time's Up!" overlay with score, accuracy, and hit/miss stats when time expires, removing the terminal print; clicks will be ignored but Q/Esc will trigger a clean quit via a quit_requested flag, with difficulty and replay options left for Task 3.

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)

```
  └ Added 2 lines
     6
     7  WHITE = (255, 255, 255)
     8  RED = (220, 60, 60)
     9 +LIGHT_GRAY = (180, 180, 190)
    10 +OVERLAY = (0, 0, 0, 180)
    11
    12  class GameEngine:
    13      def __init__(self, width, height):
```

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)

```
  └ Added 2 lines
    25          self.misses = 0
    26          self.score = 0
    27          self.font = pygame.font.SysFont("Arial", 26)
    28 +        self.big_font = pygame.font.SysFont("Arial", 56, bold=T
       +rue)
    29          self.game_over = False
    30 +        self.quit_requested = False  # set on the Game Over scr
       +een; main.py exits
    31
    32      def _spawn_target(self):
    33          x = random.randint(self.margin, self.width - self.margi
       n)
```

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)

```
  └ Added 3 lines
    36
    37      def handle_event(self, event):
    38          if self.game_over:
    39 +            # Wait on the Game Over screen; clicks are ignored.
    40 +            if event.type == pygame.KEYDOWN and event.key in (p
       +ygame.K_q, pygame.K_ESCAPE):
    41 +                self.quit_requested = True
    42              return
    43          if event.type == pygame.MOUSEBUTTONDOWN:
    44              self._handle_click(event.pos)
```

```
●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
  └ Added 4 lines, removed 3 lines
     78          return round(100 * self.hits / total, 1)
     79
     80      def render(self, screen):
     81 -        r = self.target.drawn_radius()
     82 -        pygame.draw.circle(screen, RED, (self.target.x, self.ta
        -rget.y), r)
     83 -        pygame.draw.circle(screen, WHITE, (self.target.x, self.
        -target.y), r, 2)
     81 +        if not self.game_over:
     82 +            r = self.target.drawn_radius()
     83 +            pygame.draw.circle(screen, RED, (self.target.x, sel
        +f.target.y), r)
     84 +            pygame.draw.circle(screen, WHITE, (self.target.x, s
        +elf.target.y), r, 2)
     85
     86          score_text = self.font.render(f"Score: {self.score}", T
         rue, WHITE)
     87          screen.blit(score_text, (10, 10))

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
  └ Added 21 lines, removed 4 lines
     93          acc_text = self.font.render(f"Accuracy: {self.accuracy
         ()}%", True, WHITE)
     94          screen.blit(acc_text, (self.width // 2 - 90, 10))
     95
     96 -        if self.game_over and not getattr(self, "_game_over_lo
        -gged", False):
     97 -            # NOTE: no proper game-over screen yet - see Task
        -2 in the README.
     98 -            print(f"Time's up! Final score: {self.score}  Accu
        -racy: {self.accuracy()}%")
     99 -            self._game_over_logged = True
     96 +        if self.game_over:
     97 +            self._render_game_over(screen)
     98 +
     99 +    def _render_game_over(self, screen):
```

```
    100 +        # Dim the playfield, then show the final results cente
        +red on screen.
    101 +        overlay = pygame.Surface((self.width, self.height), py
        +game.SRCALPHA)
    102 +        overlay.fill(OVERLAY)
    103 +        screen.blit(overlay, (0, 0))
    104 +
    105 +        lines = [
    106 +            (self.big_font, "Time's Up!", WHITE),
    107 +            (self.font, f"Final Score: {self.score}", WHITE),
    108 +            (self.font, f"Accuracy: {self.accuracy()}%", WHITE
        +),
    109 +            (self.font, f"Hits: {self.hits}   Misses: {self.mi
        +sses}", LIGHT_GRAY),
    110 +            (self.font, "Press Q or Esc to quit", LIGHT_GRAY),
    111 +        ]
    112 +        y = self.height // 2 - 120
    113 +        for font, text, color in lines:
    114 +            surf = font.render(text, True, color)
    115 +            screen.blit(surf, surf.get_rect(midtop=(self.width
        + // 2, y)))
    116 +            y += surf.get_height() + 16
```

● Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\main.py)

```
  └ Added 2 lines
     27              if event.type == pygame.QUIT:
     28                  running = False
     29              engine.handle_event(event)
     30 +        if engine.quit_requested:
     31 +            running = False
     32
     33          engine.handle_input()
     34          engine.update()
```

● Now the headless tests. Besides the engine checks, this script runs the real main.py loop with simulated frames and keypresses, and saves a screenshot of the Game Over screen.

● Everything passes. I'll look at the screenshot myself to check the layout,

since passing tests don't show what it looks like:

Read 1 file (ctrl+o to expand)

● The screen looks right: the HUD is dimmed, the results are centred, and the frozen target is gone. Here is the change waiting to be committed:

Ran 1 shell command (ctrl+o to expand)

● I've built Task 2 and all 15 new automated checks passed, plus the 19 Task 1 checks still pass. I haven't committed it.

What changed (Lab repo only):
- game/game_engine.py:
  - When the timer reaches 0, the frozen target disappears and the screen dims. A centred panel shows Time's Up!, the final score, the accuracy, hits and misses, and "Press Q or Esc to quit".
  - The old terminal print is gone.
  - On that screen, clicks and other keys are ignored and the stats can't change. Q or Esc asks the game to quit.
- main.py: 2 lines that end the loop cleanly when quit is requested. Closing the window works as before.
- Q and Esc do nothing during play, only on the Game Over screen.

What the tests covered:
- The timer reaching 0 brings up the Game Over screen, and nothing is printed to the terminal.
- After that, more frames and clicks don't change the score or accuracy.
- Q and Esc quit, and other keys don't.
- Normal play still works.
- The real main.py loop reached Game Over at frame 5, stayed on that screen, and exited on Q at frame 61. It exited because of Q, not the test's safety cut-off.
- I looked at the saved screenshot to confirm the layout. Above is how it renders headless; fonts may look slightly different on your screen.

Please check it by hand:
cd C:\Users\rohit\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer
python main.py
1. Play for the full 30 seconds. The Game Over screen should appear with your

score and accuracy, and nothing should print in the terminal.
2. Click around on that screen. Nothing should change.
3. Press Q or Esc. The game should close.
4. Optional: start it again and press Q mid-game. It should keep playing.

Once you confirm, I'll commit it as Task 2: Add game over screen. Replay and difficulty are left for Task 3.

I manually tested Task 2, and all three checks passed. The Game Over screen displays the final statistics, clicks and other keys do not change the results, and Q/Esc close the game.

Please commit Task 2 as a separate commit with a clear message. Then verify the commit hash, commit history, and Git status, and show me the results. Do not push anything yet. Stop after the verification and wait for my confirmation before starting Task 3.

Ran 1 shell command (ctrl+o to expand)

● Task 2 is committed in your Lab repo. Nothing has been pushed.

The commit:
- Hash: 8a5564e4bd6c3da07bbb6f0267a9b26f296e61aa (short form 8a5564e)
- Message: Task 2: Add game over screen
- Author: Rohith Karning, with the Co-Authored-By: Claude line.
- Files: only game/game_engine.py and main.py (34 lines added, 7 removed).

History:
8a5564e Task 2: Add game over screen
03cdc5f Task 1: Match hit area to shrinking target radius
4e04a46 Lab 4: Add original Target Aim Trainer starter code
96c1458 Completed Lab 3

Status: the working tree is clean. It's 3 commits ahead of your GitHub copy, none of them pushed. The remote is still only rohithkarning-debug/PES1UG24AM229_SE_LAB.

Both checks were done before this commit: the 15 automated checks passed and you confirmed the manual run.

I'll wait for your go-ahead before starting Task 3.

Confirmed, proceed with Task 3

● Plan for Task 3, keeping the original game as the default:

Difficulty	Start size	Smallest size	Lifespan	Key
Easy	50 px	20 px	150 frames (2.5 s)	1 or E
Medium (original game)	40 px	12 px	90 frames (1.5 s)	2 or M
Hard	30 px	8 px	60 frames (1.0 s)	3 or H

- The first round is Medium, so the game starts exactly as it did originally.
- On the Game Over screen, the difficulty keys start a fresh round: score, misses, accuracy and the 30-second timer all reset. Q or Esc still exits.

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
⎿ Added 32 lines, removed 9 lines
9 LIGHT_GRAY = (180, 180, 190)
10 OVERLAY = (0, 0, 0, 180)

```
    11
    12 +# Target settings per difficulty. Medium matches the original g
      +ame.
    13 +DIFFICULTIES = {
    14 +    "Easy":   {"base_radius": 50, "min_radius": 20, "lifespan_f
      +rames": 150},
    15 +    "Medium": {"base_radius": 40, "min_radius": 12, "lifespan_f
      +rames": 90},
    16 +    "Hard":   {"base_radius": 30, "min_radius": 8,  "lifespan_f
      +rames": 60},
    17 +}
    18 +
    19 +# Keys that start a new round from the Game Over screen.
    20 +DIFFICULTY_KEYS = {
    21 +    pygame.K_1: "Easy",   pygame.K_e: "Easy",
    22 +    pygame.K_2: "Medium", pygame.K_m: "Medium",
    23 +    pygame.K_3: "Hard",   pygame.K_h: "Hard",
    24 +}
    25 +
    26  class GameEngine:
    13 -    def __init__(self, width, height):
    27 +    def __init__(self, width, height, difficulty="Medium"):
    28          self.width = width
    29          self.height = height
    30
    31          self.margin = 60
    32          self.hud_height = 60
    19 -        self.target = self._spawn_target()
    20 -
    33          self.round_seconds = 30
    34 +
    35 +        self.font = pygame.font.SysFont("Arial", 26)
    36 +        self.big_font = pygame.font.SysFont("Arial", 56, bold=T
      +rue)
    37 +        self.quit_requested = False  # set on the Game Over scr
      +een; main.py exits
    38 +
    39 +        self.reset(difficulty)
    40 +
    41 +    def reset(self, difficulty):
```

```
        # Start a fresh round at the chosen difficulty.
        self.difficulty = difficulty
        self.target = self._spawn_target()
        self.time_left_frames = self.round_seconds * 60

        self.hits = 0
        self.misses = 0
        self.score = 0
-       self.font = pygame.font.SysFont("Arial", 26)
-       self.big_font = pygame.font.SysFont("Arial", 56, bold=True)
        self.game_over = False
-       self.quit_requested = False  # set on the Game Over screen; main.py exits

    def _spawn_target(self):
        x = random.randint(self.margin, self.width - self.margin)
        y = random.randint(self.margin + self.hud_height, self.height - self.margin)
-       return Target(x, y)
+       return Target(x, y, **DIFFICULTIES[self.difficulty])

    def handle_event(self, event):
        if self.game_over:
            # Wait on the Game Over screen; clicks are ignored.
-           if event.type == pygame.KEYDOWN and event.key in (pygame.K_q, pygame.K_ESCAPE):
-               self.quit_requested = True
+           if event.type == pygame.KEYDOWN:
+               if event.key in DIFFICULTY_KEYS:
+                   self.reset(DIFFICULTY_KEYS[event.key])
+               elif event.key in (pygame.K_q, pygame.K_ESCAPE):
+                   self.quit_requested = True
            return
        if event.type == pygame.MOUSEBUTTONDOWN:
            self._handle_click(event.pos)
```

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
  ⎿ Added 3 lines, removed 2 lines
    129   (self.big_font, "Time's Up!", WHITE),
    130   (self.font, f"Final Score: {self.score}", WHITE),
    131   (self.font, f"Accuracy: {self.accuracy()}%", WHITE
        ),
    132 -   (self.font, f"Hits: {self.hits}   Misses: {self.mi
        -sses}", LIGHT_GRAY),
    132 +   (self.font, f"Hits: {self.hits}   Misses: {self.mi
        +sses}   ({self.difficulty})", LIGHT_GRAY),
    133 +   (self.font, "Play again: 1 Easy   2 Medium   3 Ha
        +rd", WHITE),
    134   (self.font, "Press Q or Esc to quit", LIGHT_GRAY),
    135   ]
    135 -   y = self.height // 2 - 120
    136 +   y = self.height // 2 - 150
    137   for font, text, color in lines:
    138   surf = font.render(text, True, color)
    139   screen.blit(surf, surf.get_rect(midtop=(self.width
        // 2, y)))

● Now the Task 3 test script:

● I've built Task 3 and all 23 new automated checks passed. The Task 1 and Task
  2 tests still pass too. I haven't committed it.

  What changed (only game/game_engine.py):
  - Difficulty table:

	Start size	Smallest size	Target lifespan
Easy	50 px	20 px	2.5 s
Medium	40 px	12 px	1.5 s
Hard	30 px	8 px	1.0 s
    Medium uses exactly the original game's values.
  - New reset(difficulty) method: sets a fresh 30-second timer, puts score, hits
    and misses back to 0, and spawns a new target. The first round is Medium,
    so the game starts as it did originally.

- Game Over screen:
  - Shows which difficulty you just played and the line "Play again: 1 Easy 2 Medium 3 Hard".
  - 1/E, 2/M or 3/H starts a new round. Q or Esc still quits.
  - Difficulty keys do nothing during play.

What the tests covered:
- All 6 difficulty keys start a fully reset round with the right target settings.
- Every new target in that round keeps the difficulty, whether after a hit or a timeout.
- Easy targets last exactly 150 frames and Hard targets 60.
- Shrinking and the Task 1 click check work on the new sizes.
- Q and Esc still quit.
- The real main.py loop: a round ended, 3 was pressed, a Hard round played and ended, then Q exited.
- I checked the screenshot above: the layout fits and nothing overlaps.

Please check it by hand:
cd C:\Users\rohit\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer
python main.py
1. Finish a round, then press 1 (Easy). The score, accuracy and timer should reset, and targets should be bigger and last longer.
2. Finish that round, then press 3 (Hard). Targets should be small and disappear quickly.
3. Press 2 (Medium). It should feel like the original game.
4. Press Q or Esc on the Game Over screen. The game should close.

Once you confirm, I'll commit it as Task 3: Add replay with difficulty selection.

I manually tested Task 3, and everything works correctly. Easy, Medium and Hard behave as expected, the round statistics reset correctly, Tasks 1 and 2 still work, and Q/Esc exits from Game Over.

Please commit Task 3 as a separate commit with the message Task 3: Add replay with difficulty selection. Verify the commit hash, commit history and Git status. Do not push anything yet, and stop before starting Task 4 until I

confirm.

Ran 1 shell command (ctrl+o to expand)

● Task 3 is committed in your Lab repo. Nothing has been pushed.

The commit:
- Hash: 54bb83d0410569460797c02a16a96243f1b8a3bb (short form 54bb83d)
- Message: Task 3: Add replay with difficulty selection
- Author: Rohith Karning, with the Co-Authored-By: Claude line.
- Files: only game/game_engine.py (35 lines added, 11 removed).

History:
54bb83d Task 3: Add replay with difficulty selection
8a5564e Task 2: Add game over screen
03cdc5f Task 1: Match hit area to shrinking target radius
4e04a46 Lab 4: Add original Target Aim Trainer starter code
96c1458 Completed Lab 3

Status: the working tree is clean. It's 4 commits ahead of your GitHub copy, none of them pushed. The remote is still only rohithkarning-debug/PES1UG24AM229_SE_LAB.

Both checks were done before this commit: the 23 automated checks passed (with Tasks 1 and 2 still passing), and you confirmed the manual run.

I'll wait for your go-ahead before starting Task 4.

Confirmed, proceed with Task 4

● Plan for Task 4:
- New file game/sounds.py: creates three short sounds in code, so no audio files and no new packages.
  - Hit: a short high "blip".
  - Miss: a short low buzz. A target timing out uses the same sound.
  - Round end: three descending notes, played once when the timer reaches 0.
- If audio isn't available (no sound device, mixer error), the game prints one notice and keeps running silently.
- main.py: asks for a smaller audio buffer, so sounds play without a

noticeable delay.

● Write(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\sounds.py)
⎿ Wrote 53 lines to
..\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\sounds.py
1 import array
2 import math
3 import pygame
4
5 # Sound Effects
7 # Short tones are generated in code, so the game needs no audio f
iles.
8 # If audio can't be initialized (no device, missing driver, etc.)
the
9 # game keeps running silently.
… +48 lines (ctrl+o to expand)

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
⎿ Added 1 line
1 import pygame
2 import random
3 from .target import Target
4 +from .sounds import Sounds
5
6 # Game Engine
7

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
⎿ Added 1 line
36 self.font = pygame.font.SysFont("Arial", 26)
37 self.big_font = pygame.font.SysFont("Arial", 56, bold=T
rue)
38 self.quit_requested = False # set on the Game Over scr
een; main.py exits
39 + self.sounds = Sounds()
40
41 self.reset(difficulty)
42

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
└ Added 2 lines
```
74              self.hits += 1
75              self.score += 1
76              self.target = self._spawn_target()
77 +            self.sounds.play("hit")
78          else:
79              self.misses += 1
80 +            self.sounds.play("miss")
81
82      def handle_input(self):
83          # Reserved for continuously-held-key input; this game i
        s
```

●
Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\game\game_engine.py)
└ Added 2 lines
```
91          self.time_left_frames -= 1
92          if self.time_left_frames <= 0:
93              self.game_over = True
94 +            self.sounds.play("round_end")
95              return
96
97          self.target.update()
98          if self.target.expired():
99              self.misses += 1  # letting a target time out coun
        ts as a miss too
100             self.target = self._spawn_target()
101 +           self.sounds.play("miss")
102
103     def accuracy(self):
104         total = self.hits + self.misses
```

● Update(~\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer\main.py)
└ Added 1 line
```
2  from game.game_engine import GameEngine
3
4  # Initialize pygame/Start application
```

```
     5 +pygame.mixer.pre_init(44100, -16, 2, 512)  # small buffer so sou
       +nd effects play promptly
     6  pygame.init()
     7
     8  # Screen dimensions
```

● Now the Task 4 tests. They cover which sound plays for each event and how many times, plus the "audio fails to start" case:

● I've built Task 4 and all 14 new automated checks passed. The Task 1, 2 and 3 tests still pass. I haven't committed it.

What changed:
- New game/sounds.py: builds three short sounds in code, so there are no audio files and no new packages.
  - Hit: a 0.08 s high blip.
  - Miss: a 0.12 s low buzz.
  - Round end: three falling notes, 0.6 s in total.

  If audio can't start, it prints Sound disabled: <reason> once and the game runs silently.
- game/game_engine.py: plays a hit or miss sound on each click, the miss sound when a target times out, and the round-end sound once when the timer reaches 0.
- main.py: one line asking for a small audio buffer, so sounds aren't delayed.

What the tests covered:
- Each event plays the right sound exactly once. That includes a timed-out target.
- The round-end sound plays once even with 200 more frames on the Game Over screen.
- Clicks on the Game Over screen are silent.
- After a replay, the sounds work again in the new round.
- Audio failure: with the mixer forced to fail, the game didn't crash, printed one notice, and a full round plus a replay still worked.
- Your real sound device: the sounds loaded on your Windows audio device at 44100 Hz stereo. This test briefly played the three sounds through your speakers. That only proves they play without errors. Whether they sound right is for you to judge.

Please check it by hand:
cd C:\Users\rohit\PES1UG24AM229_SE_LAB\Lab-4\target-aim-trainer
python main.py
1. Hit a target. You should hear a high blip.
2. Click empty space. You should hear a low buzz.
3. Let a target shrink and disappear. You should hear the same low buzz.
4. Wait for the timer to reach 0. You should hear three falling notes once, then silence on the Game Over screen.
5. Press 1, 2 or 3 to replay. The sounds should still work. Then Q or Esc to quit.

Once you confirm, I'll commit it as Task 4: Add sound effects for hit, miss, and round end.

I manually tested Task 4. The hit, miss, target-timeout and round-end sounds all work, and sounds continue working after replay. Please commit Task 4 as a separate commit with a clear message. Verify the commit hash, history and Git status, and stop before pushing or making further changes.

Ran 1 shell command (ctrl+o to expand)

● Task 4 is committed in your Lab repo, so all four tasks are now done. Nothing has been pushed.

The commit:
- Hash: 3a5f47b4a1c5e0f5f70daa25dd079f1a5fd02c42 (short form 3a5f47b)
- Message: Task 4: Add sound effects for hit, miss, and round end
- Author: Rohith Karning, with the Co-Authored-By: Claude line.
- Files: new game/sounds.py, plus game/game_engine.py and main.py (60 lines added).